

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Мобильная разработка

Отчет

Курсовая работа

Выполнил:

Прокопец Семен

Группа К3439

Проверил:

Шатинский Г. С.

Санкт-Петербург

2025 г.

Отчет по курсовой работе

Тема: Разработка Android-приложения «Мессенджер» с использованием встроенной навигации и архитектуры MVVM.

1. Постановка задачи и стек технологий

Цель курсовой работы — спроектировать и реализовать мобильное приложение «Мессенджер» на Android

Стек: Kotlin, Android SDK, AndroidX, Navigation Component, ViewModel и LiveData, Room, Retrofit, Material Components for Android.

2. Задача 1. Главная активность и навигация по вкладкам

1) Одна главная активность. Главная Activity выступает единственной точкой входа в приложение и содержит контейнер для навигации по фрагментам. Это упрощает управление жизненным циклом и навигацией, а также позволяет централизованно настраивать общие элементы интерфейса (тема, нижняя навигация).

2) Bottom Navigation внутри активности. Внутри Activity размещена панель BottomNavigationView, которая предоставляет пользователю быстрый доступ к основным разделам приложения: лента, профиль, настройки. Назначение — обеспечить понятную и предсказуемую навигацию между ключевыми экранами.

3) Отдельные экраны по вкладкам. Для каждой вкладки создан отдельный Fragment: фрагмент новостной ленты (изначально заглушка с текстом), фрагмент профиля пользователя и фрагмент настроек. Каждый экран отвечает за свою область данных и логику, что облегчает поддержку и расширение приложения.

4) Навигация через NavController и Navigation Graph. Переходы между фрагментами описаны в графе навигации, а BottomNavigationView связан с NavController. Это позволяет декларативно описывать навигацию, управлять back-стеком и анимациями переходов стандартными средствами Android.

5) Отслеживание жизненного цикла. В Activity и во фрагментах логируются ключевые этапы жизненного цикла (onCreate, onStart, onResume, onPause, onStop, onDestroy). Это нужно для анализа поведения приложения, поиска ошибок и демонстрации понимания жизненного цикла компонентов Android.

3. Задача 2. MVVM и управление состоянием профиля и настроек

1) ViewModel для профиля и настроек. Для экрана профиля и экрана настроек созданы отдельные ViewModel, которые хранят состояние пользователя (имя, статус) и выбранную тему. Назначение — отделить бизнес-логику и данные от UI, чтобы интерфейс только отображал состояние и отправлял события.

2) Экран «Профиль». На экране профиля отображаются имя и статус пользователя, которые редактируются через EditText и сохраняются во ViewModel. При изменении текста данные во ViewModel обновляются, а при изменении данных во ViewModel UI автоматически получает актуальное состояние через LiveData. Это обеспечивает устойчивость к поворотам экрана и пересозданию фрагмента.

3) Экран «Настройки» и тема. Экран настроек содержит переключатель темы (светлая/тёмная). Состояние переключателя хранится во ViewModel, а выбранный режим применяется к приложению через AppCompatActivity. Назначение — дать пользователю контроль над внешним видом приложения и показать применение MVVM для глобального состояния.

4) Сохранение данных при повороте экрана. Так как состояние хранится во ViewModel, при повороте экрана и пересоздании фрагментов данные не теряются. Это важно для UX, поскольку пользователь не теряет введённую информацию и выбранные настройки.

5) Логи жизненного цикла ViewModel. В ViewModel фиксируются ключевые этапы жизненного цикла (создание и очищение) через методы init и onCleared. Это помогает отследить, когда именно создаются и уничтожаются объекты ViewModel, и убедиться в корректной работе архитектуры MVVM.

4. Задача 3. Загрузка сообщений из API и Room для офлайн-доступа

1) MessageRepository. Создан отдельный репозиторий, который отвечает за получение данных о сообщениях. Он инкапсулирует работу с сетью (API) и локальной базой данных (Room), предоставляя ViewModel единый источник данных. Назначение — реализовать слой данных по принципу «одного источника правды».

2) Retrofit-клиент с корутинами. Для работы с публичным API настроен Retrofit-клиент, который выполняет запросы в корутинах. Это позволяет асинхронно загружать данные без блокировки UI-потока и упрощает обработку результатов и ошибок.

3) Список сообщений на вкладке «Лента». Загруженные сообщения отображаются в RecyclerView на вкладке ленты. Каждый элемент списка показывает заголовок, текст и автора сообщения. Назначение — предоставить пользователю актуальную ленту контента в удобном формате.

4) Сохранение данных в Room. После успешной загрузки из API данные сохраняются в локальную базу данных Room. Это позволяет использовать кэшированные данные без повторных сетевых запросов и обеспечивает офлайн-доступ.

5) Работа при отсутствии сети. Если сеть недоступна, репозиторий возвращает данные из локальной базы. Назначение — сделать приложение устойчивым к проблемам с подключением и сохранить функциональность в офлайн-режиме.

6) Кнопка «Обновить». На экране ленты реализовано действие ручного обновления данных. По нажатию вызывается метод репозитория, который заново загружает сообщения из сети и обновляет базу. Это даёт пользователю контроль над актуализацией данных и демонстрирует вызов сетевых операций из UI через ViewModel.

5. Задача 4. Улучшение UI, уведомления и фоновая синхронизация

1) Кастомный RecyclerView.Adapter. Для ленты реализован адаптер, который отображает аватарку, имя и текст сообщения в кастомной разметке на основе MaterialCardView. Назначение — улучшить читаемость и внешний вид ленты, а также показать работу с собственными ViewHolder и макетами элементов.

2) Компоненты Material Design (CardView, FloatingActionButton, AppBarLayout). В экран ленты добавлены AppBarLayout с MaterialToolbar, карточки сообщений и плавающая кнопка действия. Это создаёт современный внешний вид, обеспечивает удобный доступ к основным действиям и соответствует рекомендациям Material Design.

3) Лайки сообщений. В каждом сообщении есть иконка лайка, которая меняет состояние при клике. Состояние лайка хранится во ViewModel и влияет на отображаемый значок. Назначение — продемонстрировать локальное интерактивное состояние элементов списка и реактивное обновление UI.

4) WorkManager и фоновая синхронизация (требование). В рамках задания предусмотрена настройка WorkManager для периодического обновления списка сообщений из сети в фоне. Назначение — обеспечить регулярную синхронизацию данных даже при закрытом приложении и разгрузить пользователя от ручных обновлений.

5) Уведомления о новых данных (требование). После успешной фоновой синхронизации планируется отправка Notification с текстом «Новые данные получены». Это нужно, чтобы информировать пользователя об обновлениях контента, даже если он не находится внутри приложения.

6) Запросы разрешений и реакция на онлайн/офлайн (требование). При запуске приложения предполагается запрос разрешений (уведомления, контакты и др.), а также корректная реакция на смену статуса сети. Назначение — подготовить приложение к работе с системными ресурсами и событиями, соблюдая модель разрешений Android.

6. Листинги ключевых классов и разметки

3.1. Kotlin-классы

MainActivity

```
package com.example.vsemk

import android.os.Bundle
import android.util.Log
import android.view.View
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.app.AppCompatActivity
import androidx.lifecycle.ViewModelProvider
import androidx.navigation.findNavController
import androidx.navigation.ui.setupWithNavController
import com.example.vsemk.ui.settings.SettingsViewModel
import com.google.android.material.bottomnavigation.BottomNavigationView

class MainActivity : AppCompatActivity() {

    companion object {
        private const val TAG = "MainActivity"
    }

    private lateinit var settingsViewModel: SettingsViewModel

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        settingsViewModel = ViewModelProvider(this)[SettingsViewModel::class.java]

        applyTheme(settingsViewModel.isDarkTheme.value ?: false)

        settingsViewModel.isDarkTheme.observe(this) { isDark ->
            applyTheme(isDark)
            Log.d(TAG, "Тема изменена: ${if (isDark) "тёмная" else "светлая"}")
        }

        setContentView(R.layout.activity_main)

        findViewById<View>(R.id.nav_host_fragment).post {
            val navController = findNavController(R.id.nav_host_fragment)

            findViewById<BottomNavigationView>(R.id.bottom_nav).setupWithNavController(navController)
        }

        Log.d(TAG, "onCreate")
    }

    override fun onStart() {
```

```

        super.onStart()
        Log.d(TAG, "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy")
    }

    private fun applyTheme(isDark: Boolean) {
        val mode = if (isDark) {
            AppCompatActivity.MODE_NIGHT_YES
        } else {
            AppCompatActivity.MODE_NIGHT_NO
        }
        AppCompatActivity.setDefaultNightMode(mode)
    }
}

```

MainActivity реализует основную логику управления темой приложения (светлой/тёмной) на основе данных из SettingsViewModel, используя AppCompatActivity. Активность также настраивает навигацию через NavController и связывает её с BottomNavigationView для переключения между фрагментами. Все ключевые жизненные циклы активности логируются, что упрощает отладку. Код демонстрирует корректное применение архитектурных компонентов Android (ViewModel, Navigation Component) и реактивное обновление UI при изменении настроек.

FeedFragment

```
package com.example.vsemk.ui.feed

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Observer
import androidx.recyclerview.widget.LinearLayoutManager
import com.example.vsemk.R
import com.google.android.material.snackbar.Snackbar
import com.google.android.material.floatingactionbutton.FloatingActionButton

class FeedFragment : Fragment() {

    companion object {
        private const val TAG = "FeedFragment"
    }

    private val viewModel: FeedViewModel by viewModels()
    private lateinit var adapter: MessageAdapter
    private lateinit var recyclerView: androidx.recyclerview.widget.RecyclerView
    private lateinit var refreshButton: FloatingActionButton
    private lateinit var progressBar: View
    private lateinit var errorTextView: android.widget.TextView

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "onCreateView")
        return inflater.inflate(R.layout.fragment_feed, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated")

        setupViews(view)
        setupRecyclerView()
        setupObservers()
        setupClickListeners()
    }

    private fun setupViews(view: View) {
        recyclerView = view.findViewById(R.id.recycler_view)
        refreshButton = view.findViewById(R.id.fab_refresh)
        progressBar = view.findViewById(R.id.progress_bar)
        errorTextView = view.findViewById(R.id.tv_error)
    }
}
```

```

    }

    private fun setupRecyclerView() {
        adapter = MessageAdapter { id ->
            viewModel.toggleLike(id)
        }
        recyclerView.layoutManager = LinearLayoutManager(requireContext())
        recyclerView.adapter = adapter
    }

    private fun setupObservers() {
        viewModel.messages.observe(viewLifecycleOwner, Observer { messages ->
            adapter.submitList(messages)
            Log.d(TAG, "Сообщения обновлены в UI: ${messages.size}")
        })

        viewModel.isLoading.observe(viewLifecycleOwner, Observer { isLoading ->
            progressBar.visibility = if (isLoading) View.VISIBLE else View.GONE
            refreshButton.isEnabled = !isLoading
        })

        viewModel.error.observe(viewLifecycleOwner, Observer { error ->
            if (error != null) {
                errorTextView.text = error
                errorTextView.visibility = View.VISIBLE
                Snackbar.make(requireView(), error, Snackbar.LENGTH_LONG).show()
                Log.e(TAG, "Ошибка: $error")
            } else {
                errorTextView.visibility = View.GONE
            }
        })
    }

    private fun setupClickListeners() {
        refreshButton.setOnClickListener {
            viewModel.refreshMessages()
            Log.d(TAG, "Нажата кнопка обновления")
        }
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause")
    }

```



```

    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "onDestroyView")
    }
}

```

FeedFragment отвечает за отображение ленты сообщений с использованием RecyclerView и MessageAdapter. Он реализует реактивное взаимодействие с FeedViewModel, подписываясь на изменения списка сообщений, состояния загрузки и ошибок. Поддерживается обновление данных через FloatingActionButton, при этом UI корректно реагирует на состояние загрузки и ошибки. Обработка кликов по элементам списка делегируется ViewModel для переключения лайков. Фрагмент демонстрирует чёткое разделение ответственности, соблюдение жизненного цикла и хорошую практику логирования ключевых этапов.

FeedViewModel

```

package com.example.vsemk.ui.feed

import android.app.Application
import android.util.Log
import androidx.lifecycle.AndroidViewModel
import androidx.lifecycle.LiveData
import androidx.lifecycle.MutableLiveData
import androidx.lifecycle.ViewModelScope
import com.example.vsemk.data.repository.MessageRepository
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.flow.catch
import kotlinx.coroutines.flow.flowOn
import kotlinx.coroutines.launch
import kotlinx.coroutines.withContext

class FeedViewModel(application: Application) : AndroidViewModel(application) {

    companion object {
        private const val TAG = "FeedViewModel"
    }
}

```

```

}

private val repository = MessageRepository(application)

private val _messages = MutableLiveData<List<MessageUi>>()
val messages: LiveData<List<MessageUi>> = _messages

private val _isLoading = MutableLiveData<Boolean>()
val isLoading: LiveData<Boolean> = _isLoading

private val _error = MutableLiveData<String?>()
val error: LiveData<String?> = _error

init {
    Log.d(TAG, "init: ViewModel создана")
    observeMessages()
    viewModelScope.launch {
        kotlinx.coroutines.delay(100)
        loadMessages()
    }
}

private fun observeMessages() {
    viewModelScope.launch {
        try {
            repository.getAllMessages()
                .flowOn(Dispatchers.IO)
                .catch { e ->
                    Log.e(TAG, "Ошибка при наблюдении за сообщениями", e)
                    _error.value = "Ошибка при загрузке сообщений: ${e.message}"
                    _isLoading.value = false
                }
                .collect { messageList ->
                    val current = _messages.value.orEmpty().associateBy { it.id }
                    val mapped = messageList.map { entity ->
                        val existing = current[entity.id]
                        MessageUi(
                            id = entity.id,
                            title = entity.title,
                            body = entity.body,
                            userId = entity.userId,
                            isLiked = existing?.isLiked ?: false
                        )
                    }
                    _messages.value = mapped
                    _isLoading.value = false
                    Log.d(TAG, "Получено сообщений: ${messageList.size}")
                }
        } catch (e: Exception) {
            Log.e(TAG, "Критическая ошибка при наблюдении за сообщениями", e)
            _error.value = "Ошибка при загрузке сообщений: ${e.message}"
            _isLoading.value = false
        }
    }
}

```

```

    }
}

fun loadMessages() {
    viewModelScope.launch {
        try {
            _isLoading.value = true
            _error.value = null
            withContext(Dispatchers.IO) {
                repository.loadMessages()
            }
        } catch (e: Exception) {
            Log.e(TAG, "Ошибка при загрузке сообщений", e)
            _error.value = "Ошибка при загрузке сообщений: ${e.message}"
            _isLoading.value = false
        }
    }
}

fun refreshMessages() {
    viewModelScope.launch {
        try {
            _isLoading.value = true
            _error.value = null
            withContext(Dispatchers.IO) {
                repository.refreshMessages()
            }
            Log.d(TAG, "Сообщения обновлены")
        } catch (e: Exception) {
            Log.e(TAG, "Ошибка при обновлении сообщений", e)
            _error.value = "Ошибка при обновлении сообщений: ${e.message}"
            _isLoading.value = false
        }
    }
}

fun toggleLike(messageId: Int) {
    val current = _messages.value.orEmpty()
    val updated = current.map { message ->
        if (message.id == messageId) {
            message.copy(isLiked = !message.isLiked)
        } else {
            message
        }
    }
    _messages.value = updated
}

override fun onCleared() {
    super.onCleared()
    Log.d(TAG, "onCleared: ViewModel уничтожена")
}

```

```
}
```

FeedViewModel управляет состоянием ленты сообщений, инкапсулирует логику загрузки, обновления и реакции на пользовательские действия

MessageAdapter

```
package com.example.vse.mk.ui.feed

import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import android.widget.ImageButton
import android.widget.ImageView
import android.widget.TextView
import androidx.recyclerview.widget.DiffUtil
import androidx.recyclerview.widget.ListAdapter
import androidx.recyclerview.widget.RecyclerView
import com.example.vse.mk.R

class MessageAdapter(
    private val onLikeClick: (Int) -> Unit
) : ListAdapter<MessageUi, MessageAdapter.MessageViewHolder>(MessageDiffCallback()) {

    override fun onCreateViewHolder(parent: ViewGroup, viewType: Int):
        MessageViewHolder {
        val view = LayoutInflater.from(parent.context)
            .inflate(R.layout.item_message, parent, false)
        return MessageViewHolder(view, onLikeClick)
    }

    override fun onBindViewHolder(holder: MessageViewHolder, position: Int) {
        holder.bind(getItem(position))
    }

    class MessageViewHolder(
        itemView: View,
        private val onLikeClick: (Int) -> Unit
    ) : RecyclerView.ViewHolder(itemView) {
        private val avatarImageView: ImageView = itemView.findViewById(R.id.iv_avatar)
        private val nameTextView: TextView = itemView.findViewById(R.id.tv_name)
        private val bodyTextView: TextView = itemView.findViewById(R.id.tv_body)
        private val userIdTextView: TextView = itemView.findViewById(R.id.tv_user_id)
        private val likeButton: ImageButton = itemView.findViewById(R.id.btn_like)

        fun bind(message: MessageUi) {
            nameTextView.text = message.title
            bodyTextView.text = message.body
            userIdTextView.text = "Автор: ${message.userId}"
        }
    }
}
```

```

        avatarImageView.setImageResource(R.drawable.ic_avatar_24)
        likeButton.setImageResource(
            if (message.isLiked) {
                R.drawable.ic_favorite_24
            } else {
                R.drawable.ic_favorite_border_24
            }
        )
        likeButton.setOnClickListener {
            onLikeClick(message.id)
        }
    }
}

class MessageDiffCallback : DiffUtil.ItemCallback<MessageUi>() {
    override fun areItemsTheSame(oldItem: MessageUi, newItem: MessageUi): Boolean {
        return oldItem.id == newItem.id
    }

    override fun areContentsTheSame(oldItem: MessageUi, newItem: MessageUi):
Boolean {
        return oldItem == newItem
    }
}

```

MessageAdapter реализует эффективное отображение списка сообщений в RecyclerView с использованием ListAdapter и кастомного DiffUtil.ItemCallback, что обеспечивает оптимизированное обновление UI при изменении данных

MessageUi

```
package com.example.vsemk.ui.feed
```

```

data class MessageUi(
    val id: Int,
    val title: String,
    val body: String,
    val userId: Int,
    val isLiked: Boolean
)

```

MessageRepository

```
package com.example.vsemk.data.repository
```

```

import android.content.Context
import android.net.ConnectivityManager
import android.net.NetworkCapabilities
import android.os.Build
import android.util.Log

```

```

import com.example.vsemk.data.local.AppDatabase
import com.example.vsemk.data.local.MessageEntity
import com.example.vsemk.data.remote.MessageApiService
import com.example.vsemk.data.remote.MessageResponse
import com.example.vsemk.data.remote.RetrofitClient
import kotlinx.coroutines.flow.Flow

class MessageRepository(
    private val context: Context
) {

    companion object {
        private const val TAG = "MessageRepository"
    }

    private val database = AppDatabase.getDatabase(context)
    private val messageDao = database.messageDao()
    private val apiService: MessageApiService = RetrofitClient.messageApiService

    suspend fun loadMessages() {
        try {
            if (isNetworkAvailable()) {
                Log.d(TAG, "Сеть доступна, загружаем данные из API")
                val messages = fetchMessagesFromApi()
                messageDao.insertMessages(messages)
            } else {
                Log.d(TAG, "Сеть недоступна, используем данные из БД")
            }
        } catch (e: Exception) {
            Log.e(TAG, "Ошибка при получении сообщений", e)
        }
    }

    suspend fun refreshMessages() {
        try {
            Log.d(TAG, "Обновление сообщений из API")
            val messages = fetchMessagesFromApi()
            messageDao.deleteAllMessages()
            messageDao.insertMessages(messages)
        } catch (e: Exception) {
            Log.e(TAG, "Ошибка при обновлении сообщений", e)
            throw e
        }
    }

    fun getAllMessages(): Flow<List<MessageEntity>> {
        return messageDao.getAllMessages()
    }

    private suspend fun fetchMessagesFromApi(): List<MessageEntity> {
        val response = apiService.getMessages()
        return response.map { it.toEntity() }
    }
}

```

```

private fun MessageResponse.toEntity(): MessageEntity {
    return MessageEntity(
        id = this.id,
        title = this.title,
        body = this.body,
        userId = this.userId
    )
}

private fun isNetworkAvailable(): Boolean {
    return try {
        val connectivityManager =
context.getSystemService(Context.CONNECTIVITY_SERVICE) as? ConnectivityManager
        if (connectivityManager == null) {
            Log.w(TAG, "ConnectivityManager недоступен")
            return false
        }

        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.M) {
            val network = connectivityManager.activeNetwork ?: return false
            val capabilities = connectivityManager.getNetworkCapabilities(network)
            ?: return false
            capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_INTERNET)
            &&
capabilities.hasCapability(NetworkCapabilities.NET_CAPABILITY_VALIDATED)
        } else {
            @Suppress("DEPRECATION")
            val allNetworks = connectivityManager.allNetworks
            for (net in allNetworks) {
                val networkInfo = connectivityManager.getNetworkInfo(net)
                if (networkInfo != null && networkInfo.state ==
android.net.NetworkInfo.State.CONNECTED) {
                    return true
                }
            }
            false
        }
    } catch (e: Exception) {
        Log.e(TAG, "Ошибка при проверке сети", e)
        false
    }
}
}

```

MessageRepository выступает в роли единой точки доступа к данным, объединяя работу с локальной базой данных и удалённым API. Он реализует стратегию «сеть при наличии, кэш при отсутствии»: при загрузке сообщений проверяется доступность интернета, и данные либо подтягиваются с сервера и сохраняются в БД, либо используются

уже имеющиеся локальные записи.

MessageEntity

```
package com.example.vsemk.data.local
```

```
import androidx.room.Entity
import androidx.room.PrimaryKey
```

```
@Entity(tableName = "messages")
data class MessageEntity(
    @PrimaryKey
    val id: Int,
    val title: String,
    val body: String,
    val userId: Int
)
```

ProfileFragment

```
package com.example.vsemk.ui.profile
```

```
import android.os.Bundle
import android.text.Editable
import android.text.TextWatcher
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup
import androidx.fragment.app.Fragment
import androidx.fragment.app.viewModels
import androidx.lifecycle.Observer
import com.example.vsemk.R
import com.google.android.material.textfield.TextInputEditText
```

```
class ProfileFragment : Fragment() {

    companion object {
        private const val TAG = "ProfileFragment"
    }

    private val viewModel: ProfileViewModel by viewModels()

    private lateinit var etUserName: TextInputEditText
    private lateinit var etUserStatus: TextInputEditText

    private var isUpdatingFromViewModel = false

    override fun onCreateView(
        inflater: LayoutInflater,
```



```

        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View? {
        Log.d(TAG, "onCreateView")
        return inflater.inflate(R.layout.fragment_profile, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated")

        etUserName = view.findViewById(R.id.et_user_name)
        etUserStatus = view.findViewById(R.id.et_user_status)

        viewModel.userName.observe(viewLifecycleOwner, Observer { name ->
            if (!isUpdatingFromViewModel && etUserName.text?.toString() != name) {
                isUpdatingFromViewModel = true
                etUserName.setText(name)
                isUpdatingFromViewModel = false
            }
        })

        viewModel.userStatus.observe(viewLifecycleOwner, Observer { status ->
            if (!isUpdatingFromViewModel && etUserStatus.text?.toString() != status) {
                isUpdatingFromViewModel = true
                etUserStatus.setText(status)
                isUpdatingFromViewModel = false
            }
        })

        etUserName.addTextChangedListener(object : TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int,
after: Int) {}

            override fun onTextChanged(s: CharSequence?, start: Int, before: Int,
count: Int) {}

            override fun afterTextChanged(s: Editable?) {
                if (!isUpdatingFromViewModel) {
                    viewModel.updateUserName(s?.toString() ?: "")
                }
            }
        })

        etUserStatus.addTextChangedListener(object : TextWatcher {
            override fun beforeTextChanged(s: CharSequence?, start: Int, count: Int,
after: Int) {}

            override fun onTextChanged(s: CharSequence?, start: Int, before: Int,
count: Int) {}

            override fun afterTextChanged(s: Editable?) {
                if (!isUpdatingFromViewModel) {

```

```

        viewModel.updateUserStatus(s?.toString() ?: "")
    }
}

})
}

override fun onStart() {
    super.onStart()
    Log.d(TAG, "onStart")
}

override fun onResume() {
    super.onResume()
    Log.d(TAG, "onResume")
}

override fun onPause() {
    super.onPause()
    Log.d(TAG, "onPause")
}

override fun onStop() {
    super.onStop()
    Log.d(TAG, "onStop")
}

override fun onDestroyView() {
    super.onDestroyView()
    Log.d(TAG, "onDestroyView")
}
}

```

ProfileFragment реализует двустороннюю привязку данных между UI и ProfileViewModel: изменения в полях ввода (имя и статус пользователя) передаются в ViewModel, а обновления из ViewModel отображаются в интерфейсе. Для предотвращения заикливания используется флаг isUpdatingFromViewModel, что является корректным решением при отсутствии Data Binding или более продвинутых механизмов реактивности.

SettingsFragment

```

package com.example.vsemk.ui.settings

import android.os.Bundle
import android.util.Log
import android.view.LayoutInflater
import android.view.View
import android.view.ViewGroup

```

```

import androidx.fragment.app.Fragment
import androidx.fragment.app.activityViewModels
import androidx.lifecycle.Observer
import com.example.vsemk.R
import com.google.android.material.switchmaterial.SwitchMaterial

class SettingsFragment : Fragment() {

    companion object {
        private const val TAG = "SettingsFragment"
    }

    private val viewModel: SettingsViewModel by activityViewModels()

    private lateinit var switchTheme: SwitchMaterial

    override fun onCreateView(
        inflater: LayoutInflater,
        container: ViewGroup?,
        savedInstanceState: Bundle?
    ): View {
        Log.d(TAG, "onCreateView")
        return inflater.inflate(R.layout.fragment_settings, container, false)
    }

    override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
        super.onViewCreated(view, savedInstanceState)
        Log.d(TAG, "onViewCreated")

        switchTheme = view.findViewById(R.id.switch_theme)

        viewModel.isDarkTheme.observe(viewLifecycleOwner, Observer { isDark ->
            switchTheme.isChecked = isDark
        })

        switchTheme.setOnCheckedChangeListener { _, isChecked ->
            viewModel.setDarkTheme(isChecked)
        }
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume")
    }

    override fun onPause() {
        super.onPause()
    }
}

```

```

        Log.d(TAG, "onPause")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop")
    }

    override fun onDestroyView() {
        super.onDestroyView()
        Log.d(TAG, "onDestroyView")
    }
}

```

SettingsFragment отвечает за управление настройками приложения, в частности переключение между светлой и тёмной темой. Он использует SwitchMaterial для взаимодействия с пользователем и связан с SettingsViewModel через activityViewModels(), что обеспечивает общее состояние с MainActivity.

3.2. XML-разметка

Разметка основного экрана (activity_main.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <androidx.fragment.app.FragmentContainerView
        android:id="@+id/nav_host_fragment"
        android:name="androidx.navigation.fragment.NavHostFragment"
        android:layout_width="0dp"
        android:layout_height="0dp"
        app:layout_constraintTop_toTopOf="parent"
        app:layout_constraintBottom_toTopOf="@id/bottom_nav"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:defaultNavHost="true"
        app:navGraph="@navigation/nav_graph" />

    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_nav"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintEnd_toEndOf="parent"

```

```

        app:menu="@menu/bottom_nav_menu" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Разметка главного экрана, очень важна для приложения.

Разметка ленты сообщений (fragment_feed.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.coordinatorlayout.widget.CoordinatorLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <com.google.android.material.appbar.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/ThemeOverlay.MaterialComponents.Dark.ActionBar">

        <com.google.android.material.appbar.MaterialToolbar
            android:id="@+id/toolbar_feed"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:title="@string/app_name"
            app:titleCentered="true" />

    </com.google.android.material.appbar.AppBarLayout>

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:layout_marginTop="?attr/actionBarSize">

        <androidx.recyclerview.widget.RecyclerView
            android:id="@+id/recycler_view"
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1"
            android:clipToPadding="false"
            android:padding="8dp" />

        <ProgressBar
            android:id="@+id/progress_bar"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_gravity="center"
            android:visibility="gone" />

        <TextView
            android:id="@+id/tv_error"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"

```

```

        android:layout_gravity="center"
        android:padding="16dp"
        android:textColor="@android:color/holo_red_dark"
        android:visibility="gone" />

</LinearLayout>

<com.google.android.material.floatingactionbutton.FloatingActionButton
    android:id="@+id/fab_refresh"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_margin="16dp"
    android:contentDescription="@string/app_name"
    app:layout_anchorGravity="bottom|end"
    android:src="@drawable/ic_favorite_border_24"
    app:backgroundTint="@color/purple_500"
    app:tint="@android:color/white"
    app:layout_anchor="@id/recycler_view" />

</androidx.coordinatorlayout.widget.CoordinatorLayout>

```

Размечивает ленту сообщений позволяя пользователю листать их

Разметка элемента сообщения (item_message.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<com.google.android.material.card.MaterialCardView
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:layout_margin="8dp"
    app:cardCornerRadius="12dp"
    app:cardElevation="3dp">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:padding="16dp"
        android:gravity="center_vertical">

        <ImageView
            android:id="@+id/iv_avatar"
            android:layout_width="40dp"
            android:layout_height="40dp"
            android:src="@drawable/ic_avatar_24"
            android:background="@drawable/ic_avatar_background_circle"
            android:scaleType="centerInside"
            android:padding="4dp" />

```

```

<LinearLayout
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:orientation="vertical"
    android:layout_marginStart="12dp"
    android:layout_marginEnd="8dp">

    <TextView
        android:id="@+id/tv_name"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="16sp"
        android:textStyle="bold"
        android:textColor="@android:color/black" />

    <TextView
        android:id="@+id/tv_body"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:textSize="14sp"
        android:textColor="@android:color/darker_gray"
        android:layout_marginTop="4dp" />

    <TextView
        android:id="@+id/tv_user_id"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:textSize="12sp"
        android:textColor="@color/purple_500"
        android:layout_marginTop="4dp" />

</LinearLayout>

<ImageButton
    android:id="@+id/btn_like"
    android:layout_width="40dp"
    android:layout_height="40dp"
    android:background="?attr/selectableItemBackgroundBorderless"
    android:padding="8dp"
    android:contentDescription="@string/app_name"
    android:src="@drawable/ic_favorite_border_24"
    app:tint="@color/purple_500" />

</LinearLayout>

</com.google.android.material.card.MaterialCardView>

```

Рахмечивает сообщение, помогая визуализировать его содержимое

Разметка профиля (fragment_profile.xml)

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Профиль"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="32dp" />

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Имя"
        android:textSize="16sp"
        android:layout_marginBottom="8dp" />

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_marginBottom="24dp"
        android:hint="Введите имя">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/et_user_name"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:inputType="textPersonName"
            android:maxLines="1" />

    </com.google.android.material.textfield.TextInputLayout>

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Статус"
        android:textSize="16sp"
        android:layout_marginBottom="8dp" />

    <com.google.android.material.textfield.TextInputLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Введите статус">

        <com.google.android.material.textfield.TextInputEditText
            android:id="@+id/et_user_status"
            android:layout_width="match_parent"
```



```

        android:layout_height="wrap_content"
        android:inputType="text"
        android:maxLines="3" />

</com.google.android.material.textfield.TextInputLayout>

</LinearLayout>

```

У любого человека есть профиль, а у профида разметка и это она.

Разметка настроек (fragment_settings.xml)

```

<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="24dp">

    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Настройки"
        android:textSize="24sp"
        android:textStyle="bold"
        android:layout_marginBottom="32dp" />

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="horizontal"
        android:gravity="center_vertical">

        <TextView
            android:layout_width="0dp"
            android:layout_height="wrap_content"
            android:layout_weight="1"
            android:text="Тёмная тема"
            android:textSize="16sp" />

        <com.google.android.material.switchmaterial.SwitchMaterial
            android:id="@+id/switch_theme"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content" />

    </LinearLayout>

</LinearLayout>

```

Нужна чтобы разметить настройки для приложения и

комфортного настраивания

7. Выводы по курсовой работе

В ходе выполнения курсовой работы было спроектировано и реализовано Android-приложение с использованием архитектуры MVVM, Material Design и современных библиотек Android (Room, Retrofit, Navigation, LiveData, ViewModel).

Приложение демонстрирует полную цепочку работы с данными: от получения по сети и сохранения в локальную базу до отображения в пользовательском интерфейсе и интерактивных действий (лайки, изменение профиля, смена темы).

Полученный результат может служить основой для дальнейшего развития проекта: добавления фоновой синхронизации, уведомлений, расширенной работы с профилем пользователя и других сервисов. Выполненные задачи подтверждают освоение базовых и промежуточных навыков разработки клиентских Android-приложений.